



Guida Utente promethium-batch

Elaborazioni Batch

VERSIONE 0.1

24 LUGLIO 2024

Indice

1	Job	3
1.1	Classe statistica — StatsCount	5
1.2	Operatori condizionali	5
1.3	Procedure	6
1.4	Esecuzione in <i>background</i>	6
1.5	Loop	7
2	Step	8
2.1	Elaborazione parallela	9
2.1.1	First End First Write	9
2.1.2	First Start First Write	11
2.1.3	Parallel Write	12
2.1.4	Async Single Write	12
2.1.5	Async Parallel Write	12
2.2	Sorgente dati — SourceResource	13
2.3	Destinazione dati — SinkResource	14
2.3.1	Destinazione dati — SinkBufResource	15

Introduzione

Nel mondo mainframe una elaborazione batch normalmente è divisa in più parti (step), e questi sono coordinati da uno script di controllo (JCL). Questa libreria mette a disposizione delle classi per facilitare lo sviluppo delle elaborazioni dei singoli step e di coordinare l'esecuzione degli step.

Un esempio minimale di batch con un singolo step è: dove job01 è il nome del job, step01 è il nome dello

```
public Integer call() {  
    return JCL.getInstance().job("job01")  
        .execPgm("step01", step01::run)  
        .complete();  
}
```

Listato 1: esempio minimale job

step e step01::run è il riferimento al metodo che esegue lo step.

L'esecuzione del job produce nel log un report finale del tipo:

Name	!	rc	!	Date-Time Start/Skip	!	Date-Time End	!	Lapse
step01...	!	0	!	2024-05-17T14:59:12.03243396	!	2024-05-17T14:59:12.590157083	!	00:00:00.557723123
job01....	!	0	!	2024-05-17T14:59:12.028019213	!	2024-05-17T14:59:12.590998545	!	00:00:00.562979332

Listato 2: esempio report fine esecuzione job

Capitolo 1

Job

L'esempio mostrato nell'introduzione è eccessivamente semplice, in generale lo step viene lanciato con una classe aggiuntiva che fornisce i parametri usati dallo step per pilotare l'esecuzione del codice. Al posto del nome dello step, può essere passata una classe *statistica* che fornisce il nome dello step, mette a disposizione dello step dei contatori, e produce al completamento dello step un report utente di dettaglio. Inoltre il metodo che esegue lo step può restituire un *returnCode* o non restituire nulla, in questo caso è sottinteso che viene restituito il codice di successo se non ci sono eccezioni, e un codice di errore in caso di eccezioni.

Considerando tutte le combinazioni sono possibili 8 metodi per lanciare uno step, vedi lis. 3, dove P è la generica classe di parametri, e C è la classe statistica.

```
public interface ExecPgm<S> {
    <P, C extends StatsCount> S execPgm(P p, C c, BiFunction<P, C, Integer> pgm);
    <P> S execPgm(P p, String stepName, ToIntFunction<P> pgm);
    <C extends StatsCount> S execPgm(C c, ToIntFunction<C> pgm);
    S execPgm(String stepName, IntSupplier pgm);
    <P, C extends StatsCount> S execPgm(P p, C c, BiConsumer<P, C> pgm);
    <P> S execPgm(P p, String stepName, Consumer<P> pgm);
    <C extends StatsCount> S execPgm(C c, Consumer<C> pgm);
    S execPgm(String stepName, Runnable pgm);
}
```

Listato 3: Interfaccia con i metodi di esecuzione di un programma (step)

In generale un *Job* è composto da più *Step*, e gli step successivi al primo sono eseguiti in modo condizionale. Se il primo step è terminato in errore, potrebbe non avere alcun senso l'esecuzione del secondo step. Seguendo l'impostazione dei batch mainframe/COBOL, un programma che termina con successo restituisce il valore 0, un programma che termina con una segnalazione (*warning*) restituisce il valore 4, un programma che termina in errore con valore maggiore di 4.

```
public Integer call() {
    return JCL.getInstance().job("job01")
        .execPgm("step01", step01::run)
        .cond(0,NE).execPgm("step02", step02::run)
        .cond(0,NE).execPgm("step03", step03::run)
        .complete();
}
```

Listato 4: Esempio di job con step condizionali

Un esempio di job con esecuzione condizionata è mostrato nel lis. 4, l'operatore *cond* permette di indicare le condizione per le quali il programma successivo non deve essere eseguito in funzione del codice restituito (*returnCode*) dell'ultimo programma eseguito. Lo step *step01* viene eseguito in modo incondizionato, se lo step *step01* termina con un *returnCode* diverso da zero allora lo step *step02* non viene eseguito; analogamente se lo step *step02* termina con un *returnCode* diverso da zero, lo step *step03* non viene eseguito. Se lo step *step01* termina in errore e lo step *step02* non viene eseguito, la condizione di esecuzione per lo step *step03*

viene applicata sul *returnCode* dell'ultimo programma eseguito, lo step step01, che è terminato in errore, di conseguenza lo step step03 non viene eseguito.

```
Proc<Prm1> proc1 = Proc.create((p, s) -> s
    .execPgm(p, "Step11", step11::run)
    .cond(0,NE).execPgm(p, "Step12", step12::run)
);
```

Listato 5: Esempio di definizione di una procedura con una classe parametro

È possibile raggruppare l'esecuzione di una sequenza di programmi definendo una procedura, come mostrato nel lis. 5, la procedura così definita può essere eseguita in modo simile a un programma nel *job* principale, vedi lis. 6, o all'interno di un'altra procedura.

```
public Integer call() {
    Proc<Prm1> proc1 = ...
    return JCL.getInstance().job("job01")
        .execPgm("step01", step01::run)
        .cond(0,NE).execPgm("step02", step02::run)
        .cond(0,EQ,"step01").execProc(prm1, "proc01", proc1)
        .complete();
}
```

Listato 6: Esempio uso di una procedura nel job

Finora abbiamo replicato le funzionalità presenti nei JCL mainframe. In queste librerie è presente anche qualche funzionalità aggiuntiva.

```
public Integer call() {
    return JCL.getInstance().job("job01")
        .forkPgm("sort1", this::sort1)
        .forkPgm("sort2", this::sort2)
        .join()
        .cond(0,NE).execPgm("balance", this::balance)
        .complete();
}
```

Listato 7: Esempio uso di una procedura nel job

È possibile lanciare un programma o una procedura in background in modo da eseguire più step in parallelo, nel lis. 7 vengono lanciati due step in background (sort1 e sort2), si attende che tutti i programmi in background siano finiti, e se non ci sono stati errori, viene eseguito il terzo step (balance).

Name	!	rc	!	Date-Time Start/Skip	!	Date-Time End	!	Lapse
sort1&....!		0	!	2024-05-17T15:00:58.951504	!	2024-05-17T15:01:00.799122	!	00:00:01.847618
join()....!		0	!	2024-05-17T15:00:58.951838	!	2024-05-17T15:01:00.810442	!	00:00:01.858604
sort2&....!		0	!	2024-05-17T15:00:58.952275	!	2024-05-17T15:00:59.265962	!	00:00:00.313687
balance...!		0	!	2024-05-17T15:01:00.811173	!	2024-05-17T15:01:01.065912	!	00:00:00.254739
job01.....!		0	!	2024-05-17T15:00:58.950293	!	2024-05-17T15:01:01.066175	!	00:00:02.115882

Listato 8: Esempio report fine esecuzione job con step paralleli

Come si vede dal log prodotto, lis. 8, i primi due step sono stati eseguiti in background, join ha atteso che tutti i programmi in background terminassero e ha restituito come *returnCode* il massimo tra tutti quelli restituiti dai programmi in background terminati, non essendoci errori è stato eseguito il terzo step.

Nel mondo mainframe i parametri del job (degli step) sono scalari, nella classe parametro possono essere presenti anche strutture più complesse, inoltre la classe parametro non è necessariamente solo una classe che

fornisce valori di input per gli step, ma può essere usata dagli step come contesto in cui passare valori agli step successivi. In particolare se la classe parametro `<P extends Iterable<Q>`, `Q` è possibile iterare uno step (o una procedura) usando la classe `Q` come parametro dello step (o della procedura).

```
public Integer call() {
    List<String> ls = new ArrayList<>();
    return JCL.getInstance().job("job01")
        .execPgm(ls, "list", step01::run)
        .cond(0,NE).forEachPgm(ls, s->s, step02::run)
        .complete();
}
```

Listato 9: Esempio uso di loop di un programma

Nel lis. 2 è mostrato un esempio di loop su un programma; come classe parametro viene usata una lista vuota, il primo programma alimenta la lista, e per ogni valore della lista viene eseguito il secondo programma.

Dopo questa veloce panoramica sulle funzioni disponibili per coordinare l'esecuzione degli step, vediamo le funzioni in dettaglio una per una.

1.1 Classe statistica — StatsCount

La classe statistica utente deve estendere la classe statistica base (`StatsCount`), mostrata nel lis. 10. L'utente deve indicare l'etichetta da associare la programma in esecuzione (nome dello step) nel costruttore della classe, e definire un metodo che fornisca informazioni sulla elaborazione eseguita dal programma.

```
public abstract class StatsCount {
    protected StatsCount(String name) { ... }
    protected abstract void recap(PrintWriter pw);
}
```

Listato 10: Costruttore e metodo astratto della classe statistica

1.2 Operatori condizionali

Ogni step termina con un `returnCode`, gli operatori condizionali permettono di indicare se un programma o una procedura devono essere lanciati in relazione al `returnCode` dell'ultimo step eseguito, o uno degli step precedenti eseguito.

```
public interface CondAction<T> {
    JobAction<T> cond(int cc, Cond cond);
    JobAction<T> cond(int cc, Cond cond, String stepName);
    JobAction<T> when(int cc, Cond cond);
    JobAction<T> when(int cc, Cond cond, String stepName);
}
```

Listato 11: Interfaccia con gli operatori condizionali

I metodi condizionali accettano 2 o 3 argomenti, se sono presenti solo 2 argomenti il confronto è fatto con l'ultimo programma eseguito, se sono presenti 3 argomenti, il confronto è fatto con il programma indicato nel terzo argomento. Il primo argomento è il valore che vogliamo confrontare, il secondo argomento è la relazione, vedi lis. 12, tra il valore fornito e il codice restituito dal programma eseguito.

Il metodo condizionale `cond` (copiato dagli script JCL mainframe) permette di definire la condizione per **non eseguire** il programma successivo in base al codice ritornato dal programma precedentemente eseguito. Il metodo condizionale `when`, è l'opposto di `cond`, permette di definire la condizione per **eseguire** il programma successivo in base al codice ritornato dal programma precedentemente eseguito.

```
public enum Cond {
    EQ, NE, LT, GT, LE, GE
}
```

Listato 12: Enum con le relazioni degli operatori condizionali

1.3 Procedure

Una procedura permette di raggruppare un insieme di programmi e la relativa logica condizionale di esecuzione. Sono disponibili 2 costruttori statici di una procedura, lis. 13.

```
public abstract class Proc<P> {
    public static <Q> Proc<Q> create(BiFunction<Q, JobStatus, JobStatus> route);
    public static Proc<Void> create(UnaryOperator<JobStatus> route);
}
```

Listato 13: Comandi per creare una procedura

La procedura può essere creata prima del job con il suo flusso di esecuzione, o in linea nel flusso di esecuzione. Nel flusso di esecuzione di una procedura sono disponibili tutti i comandi del flusso di esecuzione del job, anche l'esecuzione di un'altra procedura.

```
public interface ExecProc<S> {
    <P> S execProc(P p, String procName, Proc<P> proc);
    S execProc(String procName, Proc<Void> proc);
}
```

Listato 14: Comandi per lanciare una procedura

Per eseguire una procedura il comando è `execProc`, vedi lis. 14

1.4 Esecuzione in *background*

Sia i programmi che le procedure possono essere lanciati in background, vedi lis. 15,16, in questo modo è possibile eseguire più step in parallelo. I programmi lanciati in questo modo sono separati rispetto al job principale.

```
public interface ForkPgm<S> {
    <P, C extends StatsCount> S forkPgm(P p, C c, BiFunction<P, C, Integer> pgm);
    <P> S forkPgm(P p, String stepName, ToIntFunction<P> pgm);
    <C extends StatsCount> S forkPgm(C c, ToIntFunction<C> pgm);
    S forkPgm(String stepName, IntSupplier pgm);
    <P, C extends StatsCount> S forkPgm(P p, C c, BiConsumer<P, C> pgm);
    <P> S forkPgm(P p, String stepName, Consumer<P> pgm);
    <C extends StatsCount> S forkPgm(C c, Consumer<C> pgm);
    S forkPgm(String stepName, Runnable pgm);
}
```

Listato 15: Interfaccia con i metodi di esecuzione di un programma (step) in *background*

A un certo punto sarà necessario recuperare il risultato di questi programmi lanciati in background nel flusso del job principale. Questo è fatto con il comando `join`, che può essere usato in due modi, vedi lis. 17. Senza argomenti attende che tutti i programmi che sono stati lanciati in background siano terminati, e restituisce il `returnCode` massimo, con l'argomento (il nome dello step associato al programma in esecuzione) attende che sia terminato il programma indicato e restituisce il suo `returnCode`. Nel caso che il risultato di

un programma che sta girando in background non sia più necessario è possibile tentare di interrompere la sua esecuzione e ignorare il suo *returnCode* con il comando quit.

```
public interface ForkProc<S> {
    <P> S forkProc(P p, String procName, Proc<P> proc);
    S forkProc(String procName, Proc<Void> proc);
}
```

Listato 16: Interfaccia con i metodi di esecuzione di una procedura in *background*

```
public interface Joinable {
    JobStatus join();
    JobStatus join(String name);
    JobStatus quit(String name);
}
```

Listato 17: Interfaccia con i metodi controllare l'esecuzione di un programma o procedura in *background*

1.5 Loop

```
public interface LoopPgm<S> {
    <P extends Iterable<Q>, Q, C extends StatsCount> S forEachPgm(P p, Function<Q, C> c,
        BiFunction<Q, C, Integer> pgm);
    <P extends Iterable<Q>, Q> S forEachPgm(P p, Function<Q, String> name, ToIntFunction<Q> pgm);
    <P extends Iterable<Q>, Q, C extends StatsCount> S forEachPgm(P p, Function<Q, C> c, BiConsumer<Q, C> pgm);
    <P extends Iterable<Q>, Q> S forEachPgm(P p, Function<Q, String> name, Consumer<Q> pgm);
}
```

Listato 18: Interfaccia elaborazione ripetuta di un programma

```
public interface LoopProc<S> {
    <P extends Iterable<Q>, Q> S forEachProc(P p, Function<Q, String> name, Proc<Q> proc);
}
```

Listato 19: Interfaccia elaborazione ripetuta di una procedura

Capitolo 2

Step

In una elaborazione di uno step in generale abbiamo una (o più) origine dati (tipicamente un file, ma non solo), una (o più) destinazione dati (tipicamente un file, ma non solo) e un processo elaborativo che prende i dati dalla origine, li trasforma in modo da poterli scrivere nella destinazione.

Nel caso che ci siano più origini dati, è il processo elaborativo che deve decidere da quale origine leggere, e non è possibile parallelizzare il processo elaborativo.

Nel caso di una singola origine dati il processo elaborativo non richiede una logica di lettura; la lettura dalla origine dati può essere fatta dalla infrastruttura, e il processo elaborativo può essere parallelizzato. In questo caso, nelle destinazioni dati può essere mantenuta o meno la sequenzialità della origine dati (dipende dalla implementazione della parallelizzazione).

Per gestire in modo generico le sorgenti e destinazioni dati vengono usate le classi

- `SourceResource` sorgente dati
- `SinkResource` destinazione dati

Nel caso che sia presente una sola sorgente dati la parte elaborativa può produrre una tupla e sarà la infrastruttura a inviare i dati alle destinazioni, in questo caso il formato elaborativo è:

```
Pgm.from(src).into(snk1,snk2).foreach(it -> { ... });
```

Listato 20: elaborazione $1 \mapsto N$

in alternativa alla parte elaborativa oltre al valore di input vengono forniti i Consumer per scrivere direttamente in dati sulle corrispondenti destinazioni, in questo caso il formato elaborativo è: entrambi i formati

```
Pgm.from(src).into(snk1,snk2).foreach((it,wr1,wr2) -> { ... });
```

Listato 21: elaborazione writer

possono essere parallelizzati, ma solo il primo permette una implementazione che mantiene l'ordine dei dati originale.

Nel caso che siano presenti più di una sorgente dati, alla parte elaborativa vengono forniti i Supplier per leggere i dati dalle origini, e i Consumer per scrivere i dati, in questo caso il formato elaborativo è:

```
Pgm.from(src1, src2).into(snk1, snk2).proceed((rd1, rd2, wr1, wr2) -> { ... });
```

Listato 22: elaborazione reader-writer

Nel caso di una sorgente possono essere usate da 0 a 24 destinazioni, nel caso di più sorgenti, da 2 a 4, si possono avere da 0 a 12 destinazioni.

Nel caso di elaborazione sequenziale la separazione della elaborazione in una parte dedicata alla lettura dei dati, una dedicata alla scrittura dei dati e una specifica per la elaborazione dei dati, non offre particolari vantaggi.

Questa separazione è propedeutica alla elaborazione parallela. Una elaborazione sequenziale nella forma 23 può essere trasformata nella forma parallela 24, sostituendo il `forEach` in `forEachParallel`: senza

```
Pgm.from(src).into(...).forEach(...);
```

Listato 23: elaborazione sequenziale

```
Pgm.from(src).into(...).forEachParallel(numTask, ...);
```

Listato 24: elaborazione multitask

nessuna altra modifica.

2.1 Elaborazione parallela

Sono presenti più implementazioni per la elaborazione parallela, vediamo i dettagli dei vari metodi per selezionare quello più opportuno.

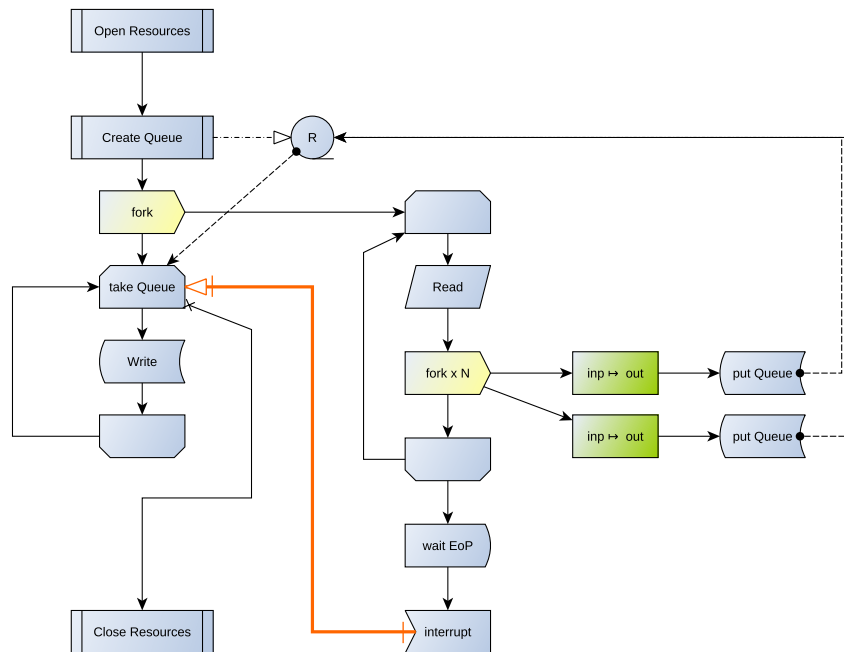


Figura 2.1: Flusso elaborativo parallelo FirstEnd-FirstWrite

2.1.1 First End First Write

Questa implementazione viene usata quando l'elaborazione parallela viene invocata con la forma 25.

L'elaborazione avviene come indicato nella fig. 2.1. Si aprono tutte le risorse, di input e di output. Si crea una coda *BlockingQueue*, che verrà usata per la gestione dell'output del processo elaborativo. Viene creato un processo secondario. Inizia un loop infinito in ascolto della coda creata, quando viene letto un dato dalla coda, in generale un tupla di N_O valori, ogni valore viene scritto sulla risorsa di output corrispondente. Quando il loop infinito viene interrotto, vengono chiuse le risorse.

```
Pgm.from(src).into(snk1,snk2).foreachParallel(numTask, it -> { ... });
```

Listato 25: elaborazione $1 \mapsto N$ (FEFW)

Torniamo al processo secondario, questo fa il loop sui dati di input. Per ogni dato letto viene avviato un task elaborativo, limitando il numero massimo di task attivi a N_T . Quando non ci sono più dati in input, si controlla che i task elaborativi siano terminati, che la coda dati sia vuota, e si invia un segnale per interrompere il loop infinito in ascolto sulla coda.

Il task elaborativo è composto dalla funzione che trasforma l'input nell'output, la λ -function del *forEach-Parallel*, e dalla scrittura dell'output sulla coda dati.

Un punto delicato di questa gestione parallela è la parte finale del processo secondario, quando sono finiti i dati di input. Dopo che sono stati sottomessi tutti i dati di input, viene invocato lo *shutdown* dello *ExecutorService* che gestisce i task elaborativi, si attende fino a un tempo limite (default 30 s), dopodiché viene invocato lo *shutdownNow* (che tenta di interrompere il task elaborativi ancora in esecuzione). Se la durata dei task elaborativi può essere superiore, è possibile impostare il valore di timeout con:

```
Pgm.from(src).into(snk1,snk2)
    .shutdownTimeout(long, TimeUnit)
    .forEachParallel(numTask, it -> { ... });
```

Listato 26: elaborazione $1 \mapsto N$ (FEFW), con shutdown timeout

Questa implementazione non mantiene l'ordine originale, l'ordine sulle risorse di output è comunque consistente. Per fare un esempio, se l'ordine dei dati in input è 1, 2, 3, sulla risorsa di output O_1 posso trovare un ordine diverso 2, 1, 3, ma tutte le risorse di output hanno lo stesso ordinamento.

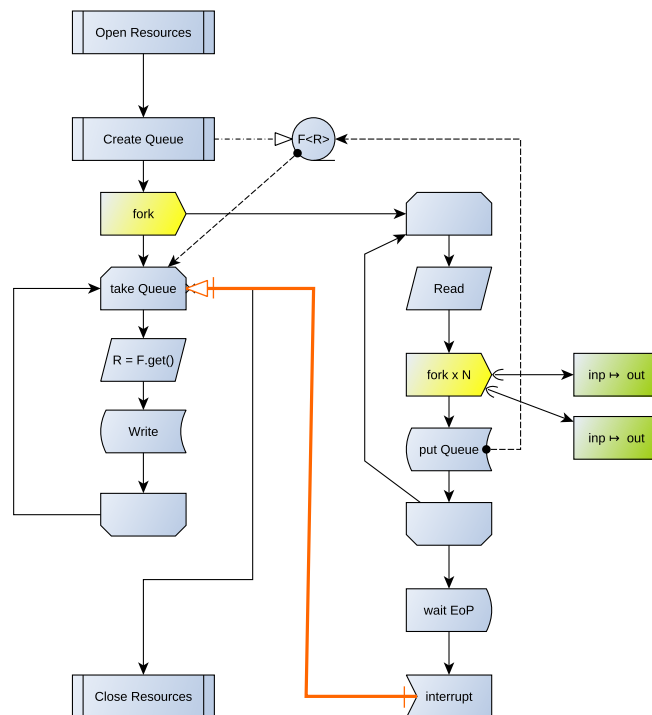


Figura 2.2: Flusso elaborativo parallelo FirstStart-FirstWrite

2.1.2 First Start First Write

Questa implementazione viene usata quando l'elaborazione parallela viene invocata con la forma 27.

```
Pgm.from(src).into(snk1,snk2).forEachParallelFair(numTask, it -> { ... });
```

Listato 27: elaborazione $1 \mapsto N$ (FSFW)

L'elaborazione avviene come indicato nella fig. 2.2. Si aprono tutte le risorse, di input e di output. Si crea una coda *BlockingQueue*, che verrà usata per la gestione dell'output del processo elaborativo. A differenza del caso precedente, § 2.1.1, la coda non ospita una tupla con gli N valori di output, ma un *Future* della tupla con gli N valori di output. Viene creato un processo secondario. Inizia un loop infinito in ascolto della coda creata, quando viene letto un *Future* dalla coda, vengono estratti gli N valori che vengono scritti sulla risorsa di output corrispondente. Quando il loop infinito viene interrotto, vengono chiuse le risorse.

Torniamo al processo secondario, questo fa il loop sui dati di input. Per ogni dato letto viene avviato un task elaborativo, limitando il numero massimo di task attivi a N_T . La sottomissione del task elaborativo restituisce un *Future* del risultato, e questo viene messo sulla coda dati. Quando non ci sono più dati in input, si controlla che i task elaborativi siano terminati, che la coda dati sia vuota, e si invia un segnale per interrompere il loop infinito in ascolto sulla coda.

Il task elaborativo è semplicemente la funzione che trasforma l'input nell'output, la λ -function del *forEachParallelFair*.

Se un task elaborativo è "lento", il loop di ascolto della coda prenderà il *Future* del risultato, ma quando tenta di estrarre il risultato, si ferma in attesa che il task termini. Se il loop di ascolto si ferma, la coda si riempie, e raggiunto un valore limite non accetta più nuovi dati. Quando il task secondario tenta di accodare il *Future* del task appena sottomesso, si ferma in attesa che la coda accetti nuovi dati, e non legge più dalla risorsa di input.

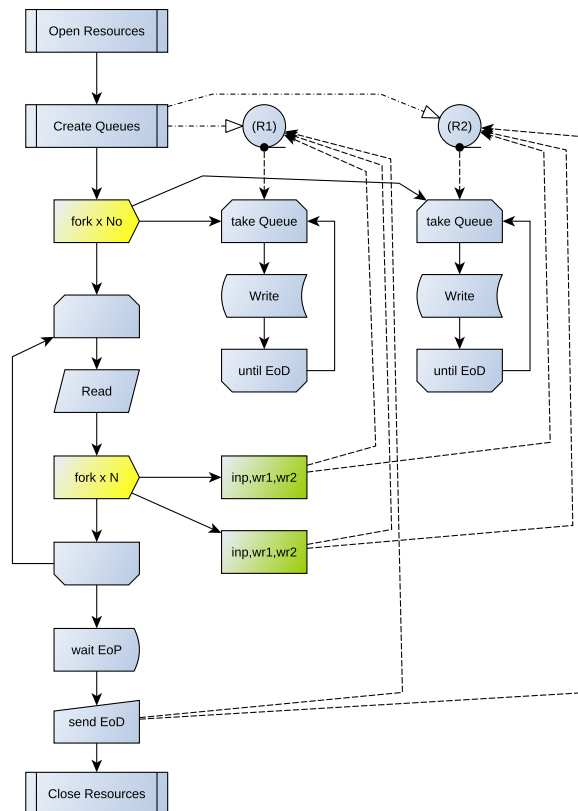


Figura 2.3: Flusso elaborativo parallelo Parallel-Write

2.1.3 Parallel Write

Questa implementazione viene usata quando l'elaborazione parallela viene invocata con la forma 28. L'ela-

```
Pgm.from(src).into(snk1,snk2).forEachParallel(numTask, (it,wr1,wr2) -> { ... });
```

Listato 28: elaborazione $1, W_1, \dots, W_N$ (PMW)

borazione avviene come indicato nella fig. 2.3. Si aprono tutte le risorse, di input e di output. Per ogni risorsa di output viene creata una coda *BlockingQueue* dedicata. Per ogni risorsa di output viene creato un task in ascolto sulla coda della rispettiva risorsa. Queste code non ospitano il tipo dati della risorsa di output, ma una classe che *Wrap*-pa del tipo dati, la classe *Wrap* permette di inviare un dato speciale che indica di terminare il loop di ascolto e il task. Inizia il loop di lettura dei dati di input. Per ogni dato letto viene avviato un task elaborativo, limitando il numero massimo di task attivi a N_T . Quando non ci sono più dati in input, si controlla che i task elaborativi siano terminati, che le code dati siano vuote, e per ogni coda dati viene inviato un valore speciale che farà terminare il task di ascolto. Si controlla che i task di ascolto siano terminati, e si chiude le risorse.

Il task elaborativo è semplicemente la λ -function del *forEachParallel*. Da osservare che la λ -function *scrive* il tipo dati della classe di output, ma sulla coda corrispondente ci finisce una versione *Wrap*-pata.

I task di ascolto sulle code, leggono il dato e lo scrivono sulla risorsa, se il dato letto è il valore speciale che indica la fine-dati, escono dal loop di ascolto e terminano il task.

Questa implementazione non mantiene l'ordine originale, inoltre l'ordine sulle risorse di output potrebbe essere inconsistente. Per fare un esempio, se l'ordine dei dati in input è 1, 2, 3, sulla risorsa di output O_1 posso trovare 2, 1, 3 e sulla risorsa di output O_2 posso trovare un altro ordine 1, 3, 2.

2.1.4 Async Single Write

Quando il processo elaborativo di trasformazione dell'input negli N valori di output è implementato mediante una funzione asincrona, lis. 29 è possibile utilizzare l'elaborazione parallela nella forma 30. Il flusso elabo-

```
@Async("TaskExecutor")
public Future<Tuple2<O1,O2>> workerTask(I i) { ... }
```

Listato 29: task $1 \mapsto N$ asincrono (ASW)

```
Pgm.from(src).into(snk1,snk2).forEachAsync(numTask, that::workerTask);
```

Listato 30: elaborazione $1 \mapsto N$ asincrona (ASW)

rativo è simile a quello della fig. 2.2, l'unica differenza è che anziché creare un *ExecutorService* per gestire la sottomissione dei task elaborativi, ne viene usato uno esterno (quello di *Spring*), e alla fine dei dati di input non viene invocato il suo *shutdown*, ma semplicemente si monitora che tutti i task elaborativi siano terminati.

Questa implementazione mantiene in output l'ordine dei dati di input.

2.1.5 Async Parallel Write

Quando il processo elaborativo che prende il valore di input e *scrive* gli N valori di output è implementato mediante una funzione asincrona, lis. 31 è possibile utilizzare l'elaborazione parallela nella forma 32. Il

```
@Async("TaskExecutor")
public Future<?> workerTask(I i, Consumer<O1> wr1, Consumer<O2> wr2) { ... }
```

Listato 31: task $1, W_1, \dots, W_N$ asincrono (APW)

```
Pgm.from(src).into(snk1,snk2).forEachAsync(numTask, that::workerTask);
```

Listato 32: Elaborazione 1, $W_1, \dots, W_N$ asincrona (APW)

flusso elaborativo è simile a quello della fig. 2.3, l'unica differenza è che anziché creare un *ExecutorService* per gestire la sottomissione dei task elaborativi, ne viene usato uno esterno (quello di *Spring*), e alla fine dei dati di input non viene invocato il suo *shutdown*, ma semplicemente si monitora che tutti i task elaborativi siano terminati.

Questa implementazione non mantiene l'ordine dei dati, vedi fine ¶ 2.1.3.

2.2 Sorgente dati — SourceResource

Una generica sorgente dati è una classe che implementa l'interfaccia *SourceResource*, vedi lis. 33, dove il metodo *get* fornisce la risorsa *U* che può essere usata in un *try-with-resources*, e i metodi *iterator* e *supplier* permettono di leggere gli elementi mediante un loop¹ o dei *supplier*².

```
public interface SourceResource<U extends AutoCloseable, I> {  
    U get();  
    Iterator<I> iterator(U u);  
    Supplier<I> supplier(U u);  
}
```

Listato 33: Metodi per aprire la risorsa e leggere i dati

L'interfaccia oltre a indicare i metodi di istanza offre dei costruttori statici facilitare l'implementazione della interfaccia.

Se leggiamo un file, una riga per volta, possiamo usare il metodo statico *bufferedReader*, sono disponibili 4 varianti, lis. 34. L'argomento *file* è obbligatorio, è possibile indicare un *charset* per la lettura del file, ed è possibile indicare un'azione da eseguire dopo aver letto una riga dal file (normalmente un contatore per le statistiche di esecuzione).

Se il file viene letto una riga per volta, e la riga viene deserializzata nella classe dati, può essere usato un costruttore nella forma lis. 35, che sono analoghi alla forma precedente, ma hanno la funzione di decodifica in modo che l'*iterator* o il *supplier* forniscano direttamente la classe dati.

Infine esistono i costruttori statici generici mediante *Iterator*, lis. 36, e *Supplier*, lis. 37.

¹uso di una *SourceResource* con un *Iterator*:

```
try (U u = source.get()) {  
    Iterator<I> iterator = source.iterator(u);  
    while (iterator.hasNext()) {  
        I i = iterator.next();  
        // TODO  
    }  
}
```

²uso di una *SourceResource* con un *Supplier*:

```
try (U u = source.get()) {  
    Supplier<I> supplier = source.supplier(u);  
    I i = supplier.get();  
    while (i != null) {  
        // TODO  
        i = supplier.get();  
    }  
}
```

```
public interface SourceResource<U extends AutoCloseable, I> {
    static SourceResource<BufferedReader, String> bufferedReader(File file, Charset cs, Runnable inc);
    static SourceResource<BufferedReader, String> bufferedReader(File file, Runnable inc);
    static SourceResource<BufferedReader, String> bufferedReader(File file, Charset cs);
    static SourceResource<BufferedReader, String> bufferedReader(File file);
}
```

Listato 34: Metodi creare una risorsa SourceResource<BufferedReader, String>

```
public interface SourceResource<U extends AutoCloseable, I> {
    static <I> SourceResource<BufferedReader, I> bufferedReader(File file, Function<String,I> dec,
                                                                Charset cs, Runnable inc);
    static <I> SourceResource<BufferedReader, I> bufferedReader(File file, Function<String,I> dec, Charset cs);
    static <I> SourceResource<BufferedReader, I> bufferedReader(File file, Function<String,I> dec, Runnable inc);
    static <I> SourceResource<BufferedReader, I> bufferedReader(File file, Function<String,I> dec);
}
```

Listato 35: Metodi creare una risorsa SourceResource<BufferedReader,I> con funzione di decodifica String $\mapsto$ I

```
public interface SourceResource<U extends AutoCloseable, I> {
    static <U extends AutoCloseable, I> SourceResource<U, I> fromIterator(Supplier<U> ctor,
                                                                           Function<U, Iterator<I>> reader);
    static <U extends AutoCloseable & Iterable<I>, I> SourceResource<U, I> fromIterator(Supplier<U> ctor);
    static <I> SourceResource<?, I> fromIterator(Iterable<I> iterable);
}
```

Listato 36: Metodi per creare una origine dati da un iterator

```
public interface SourceResource<U extends AutoCloseable, I> {
    static <U extends AutoCloseable, I> SourceResource<U, I> fromSupplier(
        Supplier<U> ctor, Function<U, Supplier<I>> reader);
    static <U extends AutoCloseable & Supplier<I>, I> SourceResource<U, I> fromSupplier(Supplier<U> ctor);
}
```

Listato 37: Metodi per creare una origine dati da un supplier

2.3 Destinazione dati — SinkResource

Una generica destinazione dati è una classe che implementa l'interfaccia SinkResource, vedi lis. 38, dove il metodo get fornisce la risorsa U che può essere usata con un try-with-resources ed il metodo accept scrive³ il dato. L'interfaccia oltre a indicare i metodi di istanza offre dei costruttori statici per facilitare l'implementazione dell'interfaccia.

Se la risorsa è un file e viene scritta una riga per volta possono essere usato il metodo statico bufferedWriter, sono disponibili 4 varianti, lis. 39. L'argomento file è obbligatorio, è possibile indicare il charset per la scrittura del file, e una azione da eseguire dopo aver scritto la riga del file (normalmente un contatore per le statistiche di esecuzione).

³uso di una SinkResource:

```
try (T t = sink.get()) {    // open resource
    O o = // TODO
    sink.accept(t, o);      // write data
}
```

```
public interface SinkResource<U extends AutoCloseable, O> {
    U get();
    void accept(U u, O o);
}
```

Listato 38: Metodi per aprire la risorsa e scrivere i dati

```
public interface SinkResource<U extends AutoCloseable, O> {
    static SinkResource<BufferedWriter, String> bufferedWriter(File file, Charset cs, Runnable inc);
    static SinkResource<BufferedWriter, String> bufferedWriter(File file, Runnable inc);
    static SinkResource<BufferedWriter, String> bufferedWriter(File file, Charset cs);
    static SinkResource<BufferedWriter, String> bufferedWriter(File file);
}
```

Listato 39: Metodi per creare una risorsa SinkResource<BufferedWriter, String>

Se la riga sul file rappresenta la serializzazione di un oggetto, possono essere usati i costruttori statici nella forma lis. 40, che sono analoghi alla forma precedente, ma hanno la funzione di codifica della classe in stringa, in modo che il *writer* possa scrivere direttamente la classe dati.

```
public interface SinkResource<U extends AutoCloseable, O> {
    static <O> SinkResource<BufferedWriter, O> bufferedWriter(File file, Function<O,String> enc, Charset cs,
        Runnable inc);
    static <O> SinkResource<BufferedWriter, O> bufferedWriter(File file, Function<O,String> enc, Charset cs);
    static <O> SinkResource<BufferedWriter, O> bufferedWriter(File file, Function<O,String> enc, Runnable inc);
    static <O> SinkResource<BufferedWriter, O> bufferedWriter(File file, Function<O,String> enc);
}
```

Listato 40: Metodi per creare una risorsa SinkResource<BufferedWriter,O> con funzione di codifica $O \mapsto \text{String}$

Infine esistono i costruttori statici generici, lis. 41. Il parametro *action* indica una azione che viene eseguita dopo la scrittura del dato (normalmente un contatore per le statistiche di esecuzione).

```
public interface SinkResource<U extends AutoCloseable, O> {
    static <U extends AutoCloseable, O> SinkResource<U, O> of(Supplier<U> ctor, BiConsumer<U, O> writer);
    static <U extends AutoCloseable, O> SinkResource<U, O> of(Supplier<U> ctor, BiConsumer<U, O> writer,
        Runnable action);
    static <O> SinkResource<?, O> of(Consumer<O> writer);
    static <O> SinkResource<?, O> of(Consumer<O> writer, Runnable action);
}
```

Listato 41: Metodi per creare una risorsa SinkResource generica

2.3.1 Destinazione dati — SinkBufResource

A volte una destinazione dati non *scrive* immediatamente i dati che riceve, ma gli accumula in un buffer, e la scrittura avviene al raggiungimento di un limite del buffer o prima della chiusura della risorsa. Questa implementazione in generale è più performante rispetto alla scrittura immediata.

Quando si hanno più destinazioni dati, in generale, le soglie dei rispettivi buffer non vengono raggiunte in modo sincrono, e i dati vengono scritti, temporaneamente, in modo inconsistente. Questo non è un problema se le risorse sono files, ma se le risorse sono update (insert) batch su tabelle diverse, collegate, di un database, potrebbe essere un problema.

L'interfaccia SinkBufResource è banalmente una estensione della interfaccia SinkResource, che rispetto a questa, ha in più il metodo `flush()` che permette di forzare la scrittura del buffer. Questo permette di impostare i legami delle risorse e scrivere i dati sulle risorse collegate in modo consistente.


```

public interface SinkBufResource<U extends AutoCloseable, O> extends SinkResource<U, O> {
    void flush();
    static <U extends AutoCloseable, O>
    SinkBufResource<U, O> of(Supplier<U> ctor, BiConsumer<U, O> writer, Consumer<U> flusher);
    static <U extends AutoCloseable, O>
    SinkBufResource<U, O> of(Supplier<U> ctor, BiConsumer<U, O> writer, Consumer<U> flusher, Runnable action);
    ...
}

```

Listato 42: Metodi per creare una risorsa SinkBufResource generica

```

public interface SinkBufResource<U extends AutoCloseable, O> extends SinkResource<U, O> {
    ...
    static SinkBufResource<BufferedWriter, String> bufferedWriter(File file, Charset cs, Runnable inc);
    static SinkBufResource<BufferedWriter, String> bufferedWriter(File file, Runnable inc);
    static SinkBufResource<BufferedWriter, String> bufferedWriter(File file, Charset cs);
    static SinkBufResource<BufferedWriter, String> bufferedWriter(File file);
    ...
}

```

Listato 43: Metodi per creare una risorsa SinkBufResource<BufferedWriter, String>

```

public interface SinkBufResource<U extends AutoCloseable, O> extends SinkResource<U, O> {
    ...
    static <O> SinkBufResource<BufferedWriter, O> bufferedWriter(
        File file, Function<O, String> enc, Charset cs, Runnable inc);
    static <O> SinkBufResource<BufferedWriter, O> bufferedWriter(
        File file, Function<O, String> enc, Runnable inc);
    static <O> SinkBufResource<BufferedWriter, O> bufferedWriter(File file, Function<O, String> enc, Charset cs);
    static <O> SinkBufResource<BufferedWriter, O> bufferedWriter(File file, Function<O, String> enc);
}

```

Listato 44: Metodi per creare una risorsa SinkBufResource<BufferedWriter, O> con funzione di codifica

Elenco dei listati

1	esempio minimale job	2
2	esempio report fine esecuzione job	2
3	Interfaccia con i metodi di esecuzione di un programma (step)	3
4	Esempio di job con step condizionali	3
5	Esempio di definizione di una procedura con una classe parametro	4
6	Esempio uso di una procedura nel job	4
7	Esempio uso di una procedura nel job	4
8	Esempio report fine esecuzione job con step paralleli	4
9	Esempio uso di loop di un programma	5
10	Costruttore e metodo astratto della classe statistica	5
11	Interfaccia con gli operatori condizionali	5
12	Enum con le relazioni degli operatori condizionali	6
13	Comandi per creare una procedura	6
14	Comandi per lanciare una procedura	6
15	Interfaccia con i metodi di esecuzione di un programma (step) in <i>background</i>	6
16	Interfaccia con i metodi di esecuzione di una procedura in <i>background</i>	7
17	Interfaccia con i metodi controllare l'esecuzione di un programma o procedura in <i>background</i>	7
18	Interfaccia elaborazione ripetuta di un programma	7
19	Interfaccia elaborazione ripetuta di una procedura	7
20	elaborazione $1 \mapsto N$	8
21	elaborazione writer	8
22	elaborazione reader-writer	8
23	elaborazione sequenziale	9
24	elaborazione multitask	9
25	elaborazione $1 \mapsto N$ (FEFW)	10
26	elaborazione $1 \mapsto N$ (FEFW), con shutdown timeout	10
27	elaborazione $1 \mapsto N$ (FSFW)	11
28	elaborazione $1, W_1, \dots, W_N$ (PMW)	12
29	task $1 \mapsto N$ asincrono (ASW)	12
30	elaborazione $1 \mapsto N$ asincrona (ASW)	12
31	task $1, W_1, \dots, W_N$ asincrono (APW)	12
32	Elaborazione $1, W_1, \dots, W_N$ asincrona (APW)	13
33	Metodi per aprire la risorsa e leggere i dati	13
34	Metodi creare una risorsa <code>SourceResource<BufferedReader, String></code>	14
35	Metodi creare una risorsa <code>SourceResource<BufferedReader, I></code> con funzione di decodifica	14
36	Metodi per creare una origine dati da un iterator	14
37	Metodi per creare una origine dati da un supplier	14
38	Metodi per aprire la risorsa e scrivere i dati	15
39	Metodi per creare una risorsa <code>SinkResource<BufferedWriter, String></code>	15
40	Metodi per creare una risorsa <code>SinkResource<BufferedWriter, 0></code> con funzione di codifica	15
41	Metodi per creare una risorsa <code>SinkResource</code> generica	15
42	Metodi per creare una risorsa <code>SinkBufResource</code> generica	16
43	Metodi per creare una risorsa <code>SinkBufResource<BufferedWriter, String></code>	16
44	Metodi per creare una risorsa <code>SinkBufResource<BufferedWriter, 0></code> con funzione di codifica	16